

TD 4 : Containers et packaging

1 «chroot» prison fragile & cgroups

La première sécurité mise en place pour assurer la protection des services est la prison **chroot**. Le **chroot** permet d'isoler les fichiers du programme à exécuter dans un sous-répertoire du système. Cela se fait en déplaçant et remplaçant la racine du système de fichier linux ("/") par un autre répertoire, c'est à dire que le répertoire courant deviendra la racine du programme que nous allons exécuter. En français, on nomme souvent cela une prison **chroot**.

Sous Debian/Ubuntu, on utilise la commande **schroot** accompagnée de la commande **debootstrap**. Car, à une date pas si lointaine, il fallait constituer à la main un mini système (ie un mini environnement) qui devait permettre l'exécution de la commande et/ou du service en copiant les fichiers utiles au programme.

Ainsi, on utilisait les commandes **strace** et **ldd** pour connaître les dépendances du programme que l'on souhaitait emprisonner et on copiait à la main l'ensemble des fichiers nécessaires.

1.1 Premier test chroot

Le but est de créer une prison pour **/usr/bin/ls** et **/usr/bin/ps**.

1.1.1 Question

A quoi servent les commandes **strace** et **ldd** ? Testez les sur un programme de votre choix ?

1.1.2 Question

Lister les bibliothèques dont dépendent de ces 2 commandes (**ls** et **ps**).

Créer la prison dans un répertoire **prison**.

Que manque-t-il pour faire fonctionner le programme **ps** ?

Comparer les exécutions de ces 2 programmes dans la prison et hors de la prison ?

Pour imposer d'autres limites, il existe les **cgroups**. Les **cgroups** sont un système de fichier virtuel qui permet d'imposer des contraintes sur des processus (et leurs enfants) à la manière de la commande shell **ulimit**.

Pour créer un environnement de contrainte sur des processus, il faut créer un répertoire dans le système de fichier virtuel.

1.1.3 Questions

Retrouver le point de montage du système de fichier virtuel **cgroup**.

Créer l'environnement **prison.slice**.

Limiter la mémoire des processus de ce **cgroup**.

1.1.4 Questions

Installer **busybox** et **bash** dans votre **prison**.

Installer les commandes busybox via **busybox --install -s**.

Allouer 900mo de memoire avec la commande shell **head -c 900000000 /dev/zero | tail**

1.1.5 Questions

Placer le shell bash précédemment lancé dans le groupe **prison.slice**

Relancer la commande pour attribuer la mémoire.

1.2 debian : des outils pour automatiser chroot

Ici, nous utiliserons les outils debian/ubuntu qui permettent de le faire automatiquement. Vous créez un fichier de configuration (par ex. **trusty.conf** dans le répertoire **/etc/schroot/chroot.d** pour fabriquer un nouveau chroot. Ensuite vous utiliserez la commande **debootstrap --variant=buildd --arch=i386 trusty /mon/dir/chroot http://archive.ubuntu.com/ubuntu/** pour remplir les fichiers nécessaires au chroot (vérifier par un simple ls).

Enfin vous utiliserez la commande **schroot -c** pour la tester d'abord en créant un fichier vide sur "/" puis en quittant la prison (exit) et en vérifiant que le fichier n'est bien présent que dans le répertoire du chroot et non dans la vraie racine de votre système. Pour terminer, vous installerez la paquetage **openvpn** dans la prison et vous testerez aussi les connexions réseaux.

Vous constatez que la SEULE protection offerte est celle du changement de racine. L'ouverture d'une socket TCP (par ex. par **netcat**) se fait avec les ressources du système de base. La protection n'est donc, dans certains cas, pas suffisante. D'où l'utilisation de conteneurs qui permettent d'isoler un peu plus les processus. Cela évite, en cas de corruption du processus, des dommages importants ou des connexions incontrôlées.

En fait, comme vu en cours, les conteneurs sont des combinaisons de **chroot**, **cgroup** et de **namespace**. Une commande plus générique que **chroot** existe dans les nouvelles distributions linux. Elle s'appelle **unshare**. Elle est exploitée dans la création des conteneurs.

2 les conteneurs podman/buildah

Nous allons donc étudier les conteneurs. Ils en existent différentes versions comme **lxc** (pour Linux Containers) ou **Dockers** ou **podman**. Les conteneurs (container) permettent une automatisation des déploiements et de l'utilisation. Ils sont un bon compromis entre la consommation faible de ressources et la faible protection qu'offre le chroot et la forte protection (mais aussi la forte consommation des ressources) des machines virtuelles.

Nous étudions, dans cette section, la suite logicielle **podman**. Regardez la liste des fichiers du paquetage **podman** / **buildah** grâce aux commandes **dpkg -L podman**, **dpkg -L buildah** et **man**. Vous aurez plus de détails sur la gestion des paquetages dans un futur td. En attendant, essayer de trouver les fichiers de configuration par défaut. De même, vous regarderez certains "patrons" (templates) de conteneur.

En ce qui concerne **podman**, il est souvent noté que pour les commandes, il suffit aussi de regarder les documentations de **docker** avec lequel **podman** est compatible. Les créateurs de **podman** indiquent d'ailleurs comme publicité : "**alias docker='podman'**".

Vous trouverez les documentations de **podman/buildah** sur les sites web suivant :

- Site de podman : <https://podman.io/>
- Site du code podman : <https://github.com/containers/podman>
- Site de la documentation de podman : <https://docs.podman.io/en/latest/>
- Site de buildah : <https://buildah.io/>
- Site du code buildah : <https://github.com/containers/buildah>

— Site de la documentation de buildah : <https://github.com/containers/buildah/tree/main/docs/tutorials>

Il faut bien retenir la terminologie. Un conteneur s'instancie à partir d'une image. Une image est une représentation d'un système de fichier complet (avec d'autres informations) figé dans une archive.

Il est possible de créer une image à partir d'un conteneur en cours d'exécution en figeant son système de fichier dans une archive de conteneur.

Ainsi dans le cadre de ce TME, nous allons tout d'abord instancier un conteneur. Pour cela vous devez obtenir une image qui servira de base à l'exécution.

2.0.1 Questions

Chercher une image debian à l'aide de la commande podman.

Récupérer cette image et créer votre premier conteneur de manière à obtenir immédiatement un shell dans ce dernier. Attention : certains conteneurs se lancent sans shell interactif à vous de le forcer.

Créer une seconde image à partir d'un dockerfile.

Comparer l'environnement d'un des conteneurs avec celui du système.

Obtenir la liste de tous les conteneurs existant (en cours d'exécution ou non). Arrêter les conteneurs et libérer leurs ressources.

Relancer un conteneur en tâche de fond (sans console). Puis une fois lancé, attachez-vous à lui via un console en exécutant un bash.

Le but de cette partie est de créer une image de conteneur puis de l'exécuter. Pour cela nous allons utiliser la commande "**buildah**". Elle permet d'instancier un conteneur de base et d'offrir des outils pour aller écrire dans le système de fichier du conteneur via les commandes "**unshare**", "**mount**", "**push**", "**pull**".

2.0.2 Questions

Créer votre base de futur image. Constaté qu'il s'agit ici d'une instance de conteneur.

Instancier un conteneur à partir de votre image.

2.0.3 Questions]

Créer un conteneur "registry". Vous laisserez la "registry" gérer les images à l'intérieur du conteneur (ie pas de point de montage d'un FS extérieur, ...).

Envoyer l'image à votre registry. Vous aurez sûrement la nécessité d'utiliser l'option "**-tls-verify=false**". Que cela sous-entend-t-il en terme de sécurité ?

3 Les conteneurs lxc

Nous allons donc étudier les conteneurs. Ils en existent différentes versions comme **lxc** (pour Linux Containers) ou **Dockers**. Les conteneurs (container) permettent une automatisation des déploiements et de l'utilisation. Ils sont un bon compromis entre la consommation faible de ressources et la faible protection qu'offre le chroot et la forte protection (mais aussi la forte consommation des ressources) des machines virtuelles.

Nous étudions, dans cette section, la suite logicielle **lxc**. Regardez la liste des fichiers du paquetage **lxc** grâce aux commandes **dpkg -L lxc** et **man**. Vous aurez plus de détails sur la gestion des paquetages dans un futur td. En attendant, essayer de trouver les fichiers de configuration par défaut. De même, vous regarderez certains "patrons" (templates) de conteneur.

Dans cette suite, on peut utiliser les commandes **lxc-ls** qui permet de lister les conteneurs, **lxc-info** qui permet d'obtenir des informations sur chaque conteneur, **lxc-start/lxc-stop** qui permettent de démarrer/stopper un conteneur, **lxc-attach/lxc-console** qui permettent de se connecter aux conteneurs, **lxc-destroy** qui permet de détruire les machines et les fichiers.

Question

Vous allez donc créer un conteneur et le lancer. Vous ne pourrez pas vous connecter la première fois car il n'y a pas de mot de passe root créer ni d'utilisateur. Vous devrez donc utiliser **lxc-attach** pour vous connecter et donner un mot de passe a root et créez un utilisateur.

4 Utilisation des outils de gestions de paquetages

Les paquets ou "package" ou paquetages sont à linux ce que sont les programmes ".exe" (ou ".msi") d'installation sont à Windows (comme par ex "setup.exe"). Ils sont extrêmement faciles à installer. Ils se présentent sous la forme de fichiers dont l'extension est ".rpm" (pour "Red-hat Package Management") sous les distributions Mandriva, RedHat, Fedora Core et Suse. Ils sont sous la forme de fichiers ".deb" pour Debian, Ubuntu etc... et sous la forme ".tgz" pour Slackware.

Pour chacune de ces distributions, il existe des outils pour gérer l'installation et la désinstallation de ces programmes (aussi appelé paquetage ou package). Les deux outils sous Mandriva sont **rpm** et **urpmi**. Il existe aussi des outils multi plateforme (comme **smart**).

RPM permet d'installer des paquets à disposition (sur un disque local). La commande rpm souffre cependant d'un petit défaut. Lorsqu'il y a des dépendances dans un paquet, il faut récupérer soit même toutes les dépendances dudit paquet pour pouvoir l'installer.

Sous Ubuntu, le gestionnaire de package local s'invoque par la commande **dpkg** qui permet d'installer, effacer, et en obtenir la liste. Pour permettre l'installation des paquetages à distance et la gestion des dépôts distant, il existe essentiellement deux suites pour cela : **aptitude** et **apt-get**.

Dès lors, d'autres commandes ont été créées pour pouvoir gérer des dépôts ("repository") de paquetages. Ces commandes permettent de déterminer automatiquement les dépendances entre les paquetages et d'installer tous les paquetages nécessaire à l'installation du logiciel souhaité.

Ces commandes sont pour Ubuntu :

- **apt** est la commande globale
- **apt-get** pour installer des sources ou des paquetages, supprimer, mettre à jour les dépôts ou mettre à jour votre distribution
- **apt-cache** pour faire des recherches (par type de paquetages, par noms, par dépendances)

Pour utiliser **apt-get**, il faut avoir le dépôts accessible. Il faut donc monter le dvd Ubuntu dans la machine virtuelle ou avoir accès au réseau. Dans le cadre de votre td, si vous voulez accéder au net, il faut utiliser la même commande que pour votre machine hôte :

```
export http_proxy=http://proxy.ufr-info-p6.jussieu.fr:3128
Defaults env_keep = "http_proxy_https_proxy_ftp_proxy"
```

Mettez à jour la distribution.

En utilisant la commande **dpkg** chercher à obtenir :

- la liste des paquetages installés
- la liste des fichiers d'un paquetage (par exemple pour le module **ntp**)
- le paquetage contenant le fichier **/etc/sudoers** item forcer la reconfiguration du paquetage du server x11

Recherche du code d'un serveur dns (bind), ses documentations et des outils associés. Les dépendances du paquetage pdns-server.

Installation du paquetage de crack de mot de passe 'lcrack'

Pour afficher la liste de tous les fichiers installé par un package on utilise la commande "dpkg". Déterminez le nom du script de démarrage de bind ainsi que le fichier de configuration associé

Supprimer le programme bind en utilisant apt-get. Quelle différence faites vous entre **remove**, **purge**, **clean**

Trouver l'équivalent du apt-get avec dpkg

5 Création de .deb

Objectif

Dans le cadre d'une gestion de parc il est intéressant de créer des fichiers rpm/deb qui permettent la mise à jour des machines. Nous allons donc créer un deb qui permet de déployer des fichiers de configurations sur votre système de manière automatique. Le deb que nous allons créer va embarquer le noyau créé précédemment et que nous nommerons "vmlinuz-compil". Le devra aussi ajouter une entrée au bootloader grub de la distribution. Nous embarquerons enfin un syslog.conf spécialement conçu pour notre système.

Installez le paquetage **opt** et regardez son contenu grâce à la commande 'ar'

Construction du paquet

Un fichier deb se construit grâce à un répertoire qui contient l'ensemble des informations nécessaire à la construction d'un fichier deb. Pour commencer, il faut créer la structure d'accueil du deb dont les répertoires sont :

- /pointdeb
 - fichier1
 - repertoire1
 - fichier2
 - ...
 - ...
- DEBIAN (repertoire METADATA)
 - control (info sur le paquetage)
 - ...

Et créer le fichier de configuration "control" contenant :

```
Package: nom_du_paquetage
Version: 0.0.1-4
Section: base
Priority: optional
Architecture: i386
Depends: unDependenceSurUneLibrairie (>= 1.2.13), uneAutreDependence (>= 1.2.6)
Maintainer: votre_nom <votre_email>
Description: Hello World
Ce programme installe les fichiers de configurations pour les machines etudiantes.
Vous y trouverez les fichiers /etc/passwd, ...
```

(ATTENTION: il doit y avoir un espace avant chaque ligne de description)

Pour obtenir des informations sur les autres META fichiers équivalents à **control** : "<https://www.debian.org/doc/manuals/maint-guide/dreq.fr.html>". Il faut ensuite collecter les fichiers à inclure dans le .deb, et les ajouter dans le répertoire racine du paquetage (/pointdeb).